

Bit Manipulation

Phase 2

- ↳ Time & Space
- ↳ Prefix Sum
- ↳ Two Pointers Sliding Window
- ↳ Sets and Maps
- ↳ Binary Search
- ↳ Number Theory Basics
- ↳ Bit Manipulation

$$\boxed{x^N}$$

$$x = 4$$

$$4^2 = 16$$

$$N = 2$$

$$x = 5$$

$$5^3 = 125$$

$$N = 3$$

$$x^N = x \times x \times x \times x \times x \times x \dots \quad (N \text{ times})$$

$$ans = 1$$

ans = 1;

```
for (i = 1; i <= N; i++)
```

```
{
```

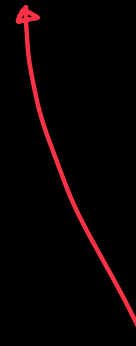
```
    ans * = x;
```

```
}
```

```
print (ans)
```

TC: $O(N)$

SC: $O(1)$



Powers of 2 $\leq N$

$$N = 20$$

1 2 4 8 16

$$N = 64$$

1 2 4 8 16 32 64

If we give you N, how many powers of 2 will be $\leq N$

$N \rightarrow$ How many powers of 2 will be $\leq N$?!

$$\underline{O(\log N)}$$

$$2^i = \underbrace{2 \times 2 \times 2 \dots \times 2}_i \text{ times}$$

$$2^{i+1} = \underbrace{2 \times 2 \times 2 \dots \times 2}_{i+1 \text{ times}}$$

$$i = 0$$

while ($\text{pow}(2, i) \leq N$)

{

print ($\text{pow}(2, i)$)

$i++$;

}

$\Downarrow$

$$N = 64$$

log N terms
will be
printed

0
1
2
:
log(N)

0
1
2
:
log(N)

① → ② → ④ → ⑧

cur = 1

1
2

Much better
method

```
cur = 1
while (cur <= N)
{
    print (cur)
    cur = cur * 2;
}
```

TC : $O(\log N)$

SC : $O(1)$

$0 + 1 + 2 + 3 + 4 + \dots + \log(N)$

Sum of N natural numbers = $N(N+1)/2$
replace N with $\log N$

This will give us all powers of 2 that are $\leq N$

$$= \frac{(\log N)(\log N + 1)}{2}$$

$$= O(\log^2 N)$$

~~$N \log N$~~

Ques 2 $N = 51 \rightarrow$ Represent N as sum of powers of 2 (No repetition allowed)

$$51 = 32 + 16 + 2 + 1$$

$$17 = 16 + 1$$

$$30 = 16 + 8 + 4 + 2$$

$$3 = 2 + 1$$

$$25 = 16 + 8 + 1$$

128

64

32

16

8

4

2

1

Find the nearest power of 2

2

$N = 51$

Find the nearest power of 2 which is closer to the number and must be $\leq$ the number

1 → 2 → 4 → 8 → 16 → 32 → 64

Find the nearest power of 2 we can take

Nearest power of 2

```
cur = 1
while (cur <= N)
{
    cur *= 2;
}
cur /= 2;
```

```
while (N > 0)
{
    if (cur <= N)
    {
        print(cur)
        N -= cur
    }
    cur /= 2;
```

32 16 8 4 2 1

TC : $O(\log N)$

SC : $O(1)$

The same code can also be used to print a number in binary form. In the if else statement, instead of printing current we can print 1 and in the else section we can print 0. Everything else will remain same

Binary Numbers

Binary Numbers → 0
→ 1

$$N = 51$$

$$51 = 32 + 16 + 2 + 1$$

32	16	8	4	2	1
1	1	0	0	1	1

Whatever we select we put a 1

$$16 + 8 + 1$$

16	8	4	2	1
1	1	0	0	1

Whatever we select we put a 1

int → 32 bits
integers can represent numbers that are 32 bits
long long → 64 bits
long long can represent numbers that are 64 bits

$$N = 25$$

32 bits int to store the number in an int variable

51 = 0 0 0 0 0 0 0 0 1 1 0 0 1 1
31 - - - - - 5 4 3 2 1 0

if there are 31 bits in an integer, we add 0's in the remaining

64 bits long long to store the number in a long variable

$$51 = \begin{array}{cccccccccccc} \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{1} & \underline{1} & \underline{0} & \underline{0} & \underline{1} & \underline{1} \\ 63 & & & & & & & & 5 & 4 & 3 & 2 & 1 & 0 \end{array}$$

$$\rightarrow 2^5 + 2^4 + 2^1 + 2^0$$

Negatives? $\rightarrow$ 2's complement $\rightarrow$ (1's complement + 1)
 we store negative number in the form of 2s complement
 Toggle every bit

$$-51 =$$

Step 1 Find the binary 51(positive)

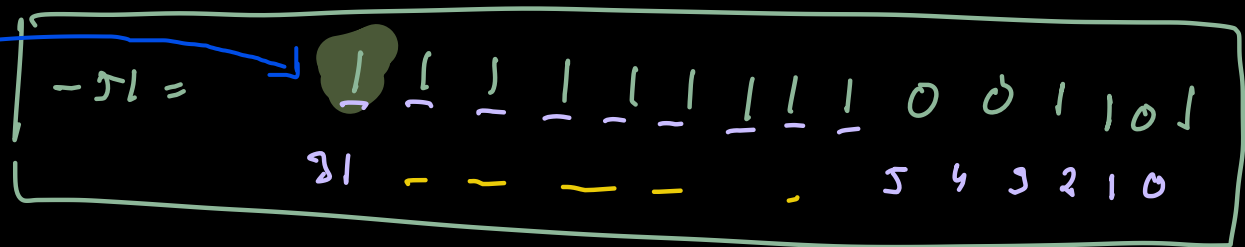
$$51 \rightarrow \begin{array}{cccccccccccc} \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{0} & \underline{1} & \underline{1} & \underline{0} & \underline{0} & \underline{1} & \underline{1} \\ 63 & & & & & & & & 5 & 4 & 3 & 2 & 1 & 0 \end{array}$$

Step 2 1's complement invert all the bits from the positive number

$$\begin{array}{cccccccccccc} \underline{1} & \underline{1} & \underline{1} & \underline{1} & \underline{1} & \underline{1} & \underline{1} & \underline{1} & \underline{0} & \underline{0} & \underline{1} & \underline{1} & \underline{0} & \underline{0} \\ 63 & & & & & & & & 5 & 4 & 3 & 2 & 1 & 0 \end{array}$$

Step 3 (+1) $\Rightarrow$ 2's complement add 1 to the previous inverted bit

This is the 2's complement



Everything in java and c++ is signed numbers

If there is 1 at the most significant bit then we cannot directly convert that to decimal

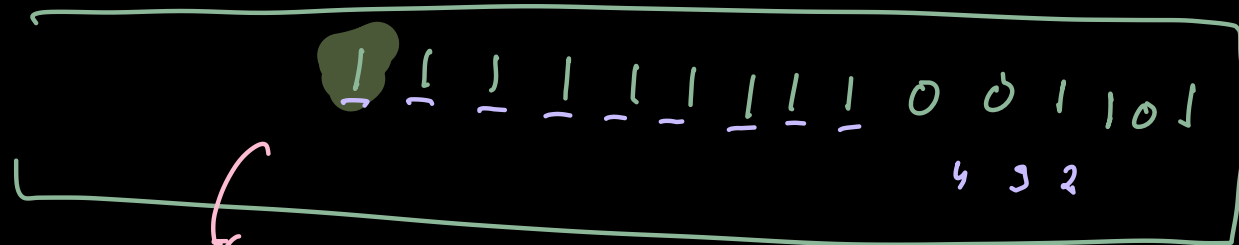
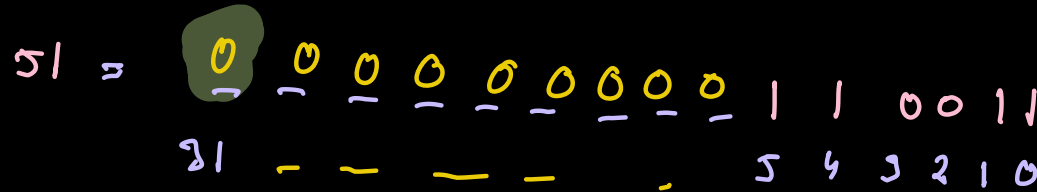
$$2^{31} + 2^{30} + 2^{29} + \dots + 2^0 \quad \times$$

Signed

Sign



0 ⇒ positive
1 ⇒ negative



1's comp

0 0 0 0 0 0 0 0 0 1 1 0 0 1 0

2's comp

+1

0 0 0 0 0 0 0 0 0 1 1 0 0 1 1

51?

Unsigned

→ (Position z_0)

Negatives are not allowed

Theory $\leftrightarrow$

$(-5) \rightarrow 00000 \dots 0101$
 $\downarrow$
 $\frac{1}{2} \quad 111111 \dots 1010$
 c
 $\Rightarrow 1 \overbrace{111111111011}^{+1}$

